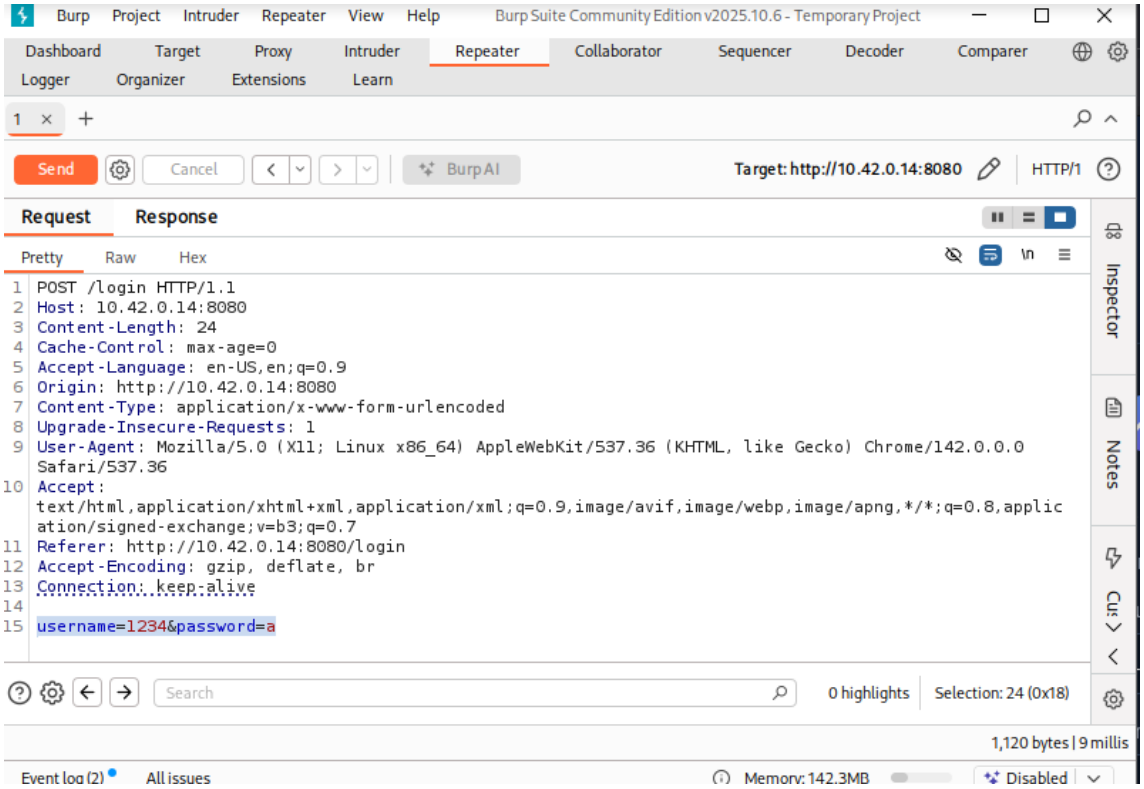
RELATÓRIO PENTEST WEB

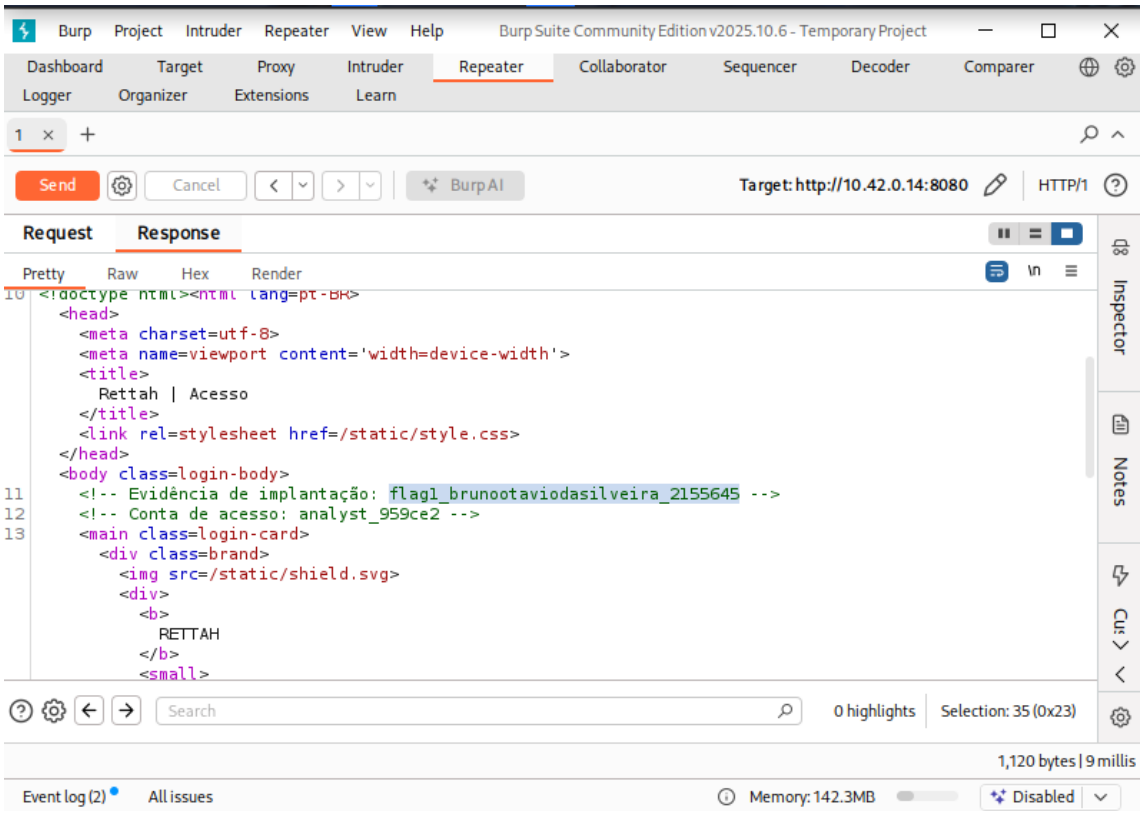
Bruno Otávio da Silveira

20/08/2026

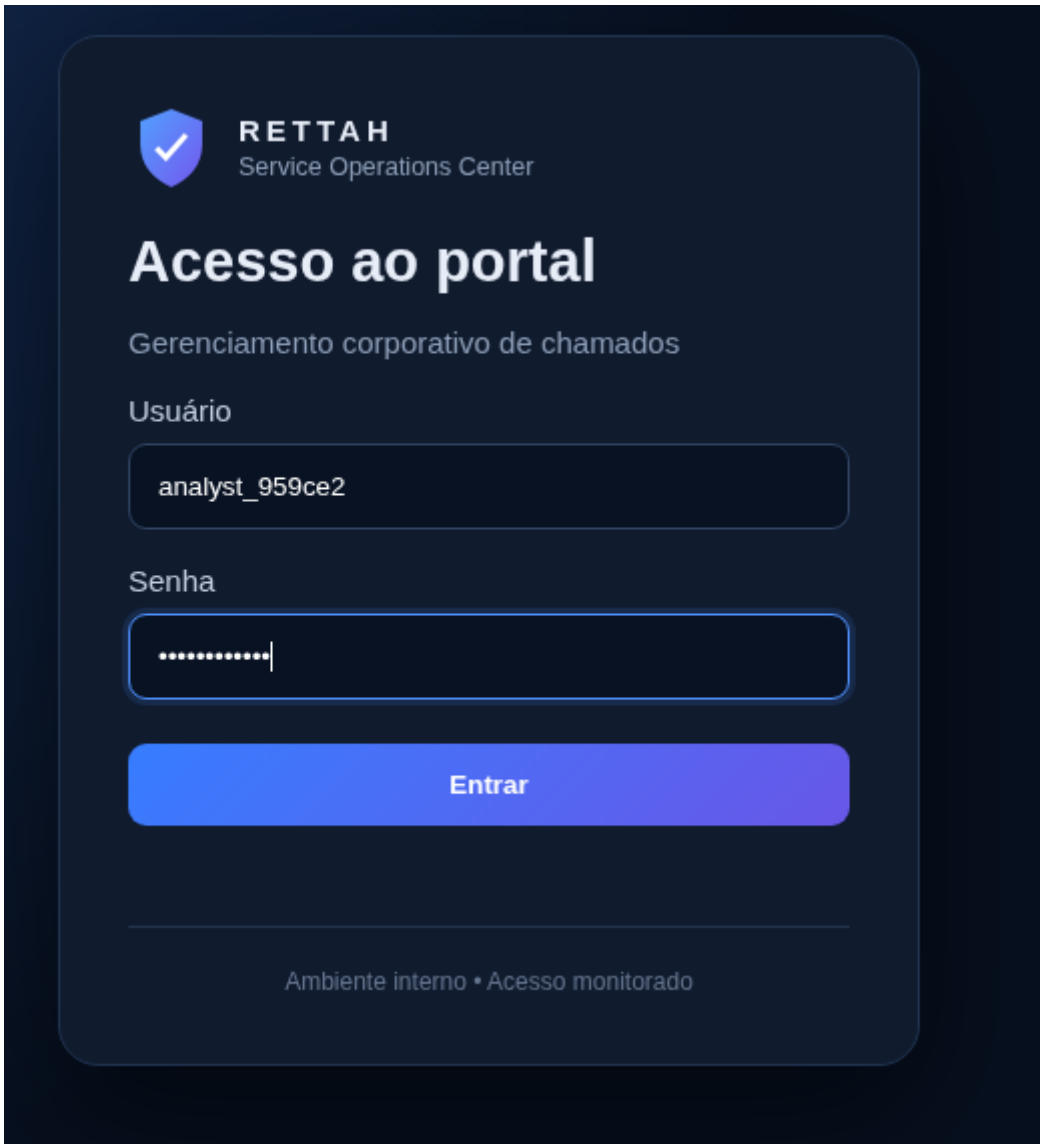
Em um primeiro momento, mandamos uma requisição usuário = 1234 e senha = 1



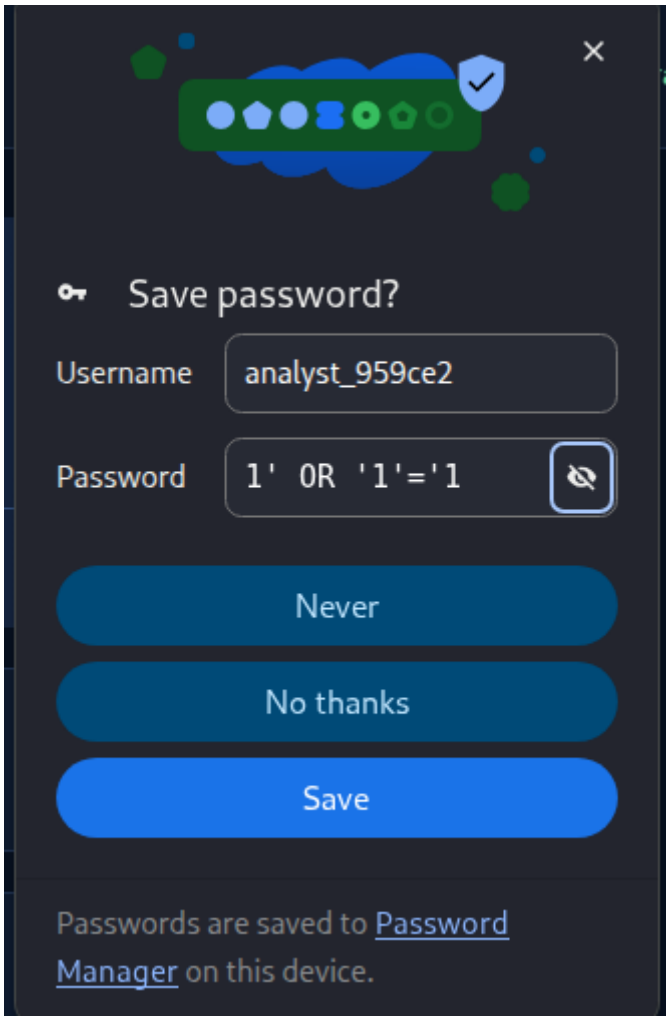
Na resposta da requisição, foi encontrada uma flag que ficou comentada no html, erro comum de programadores. Também encontramos um usuário válido no html, assim, podemos testar o sql injection na senha, já que já temos um usuário.



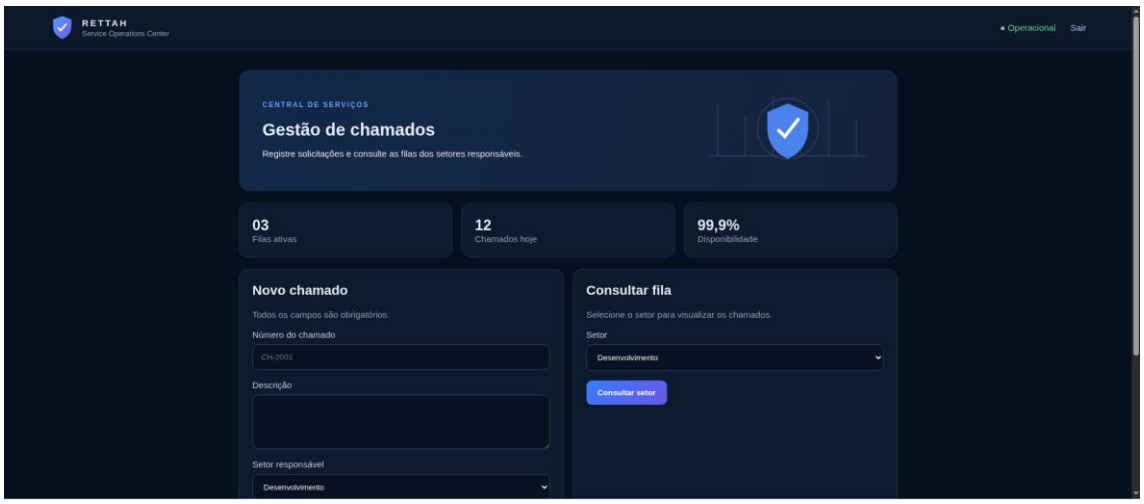
Depois disso, já que temos o usuário, vamos utilizar a injeção 1' OR '1'='1 na senha, isso irá mandar a requisição, a senha não será vazia verdadeiramente, mas 1 é igual a 1, então conseguiremos logar na aplicação


RETTAH
Service Operations Center
Acesso ao portal
Gerenciamento corporativo de chamados
Usuário
analyst_959ce2
Senha
Entrar
Ambiente interno • Acesso monitorado

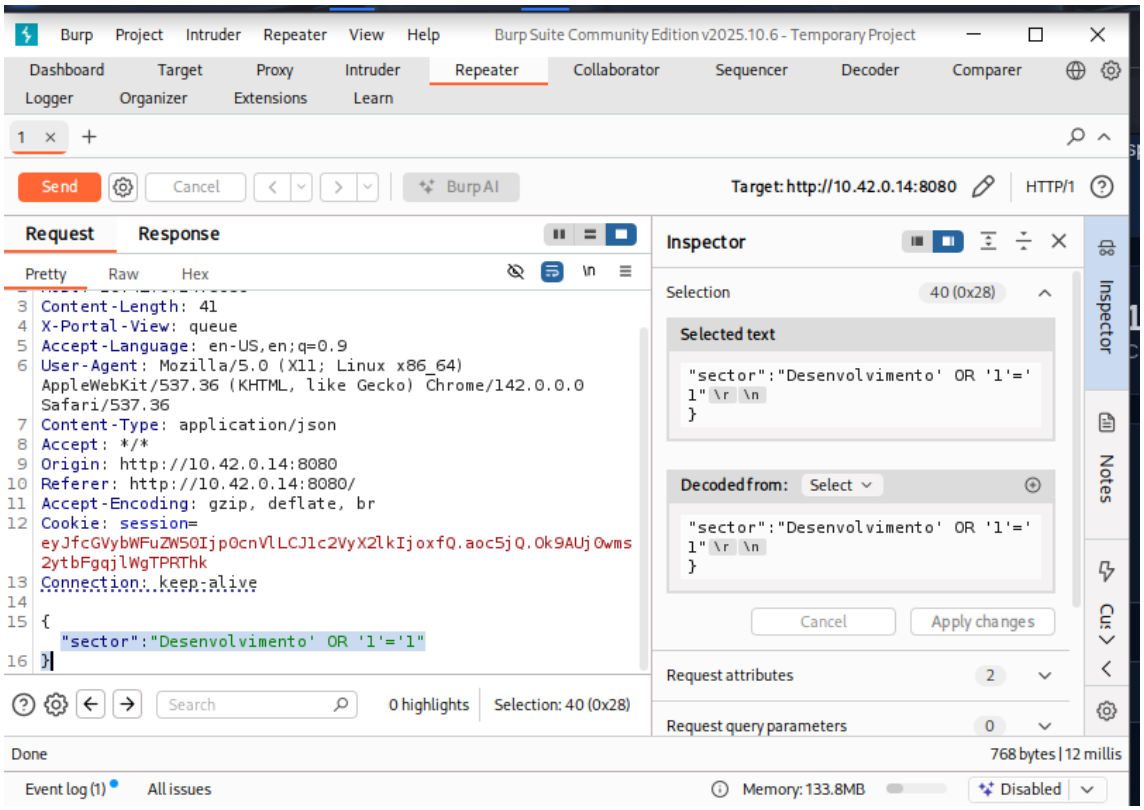
Para confirmar a senha utilizada, está ai

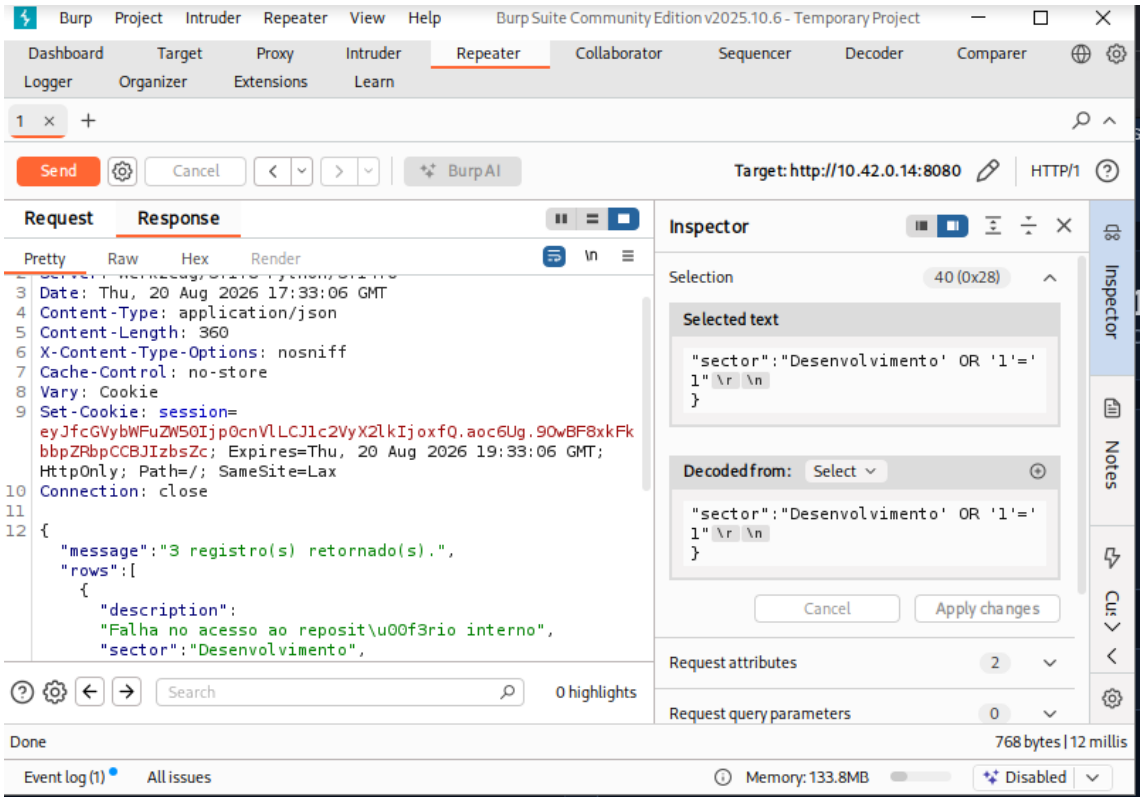


Então, entramos na página inicial da aplicação

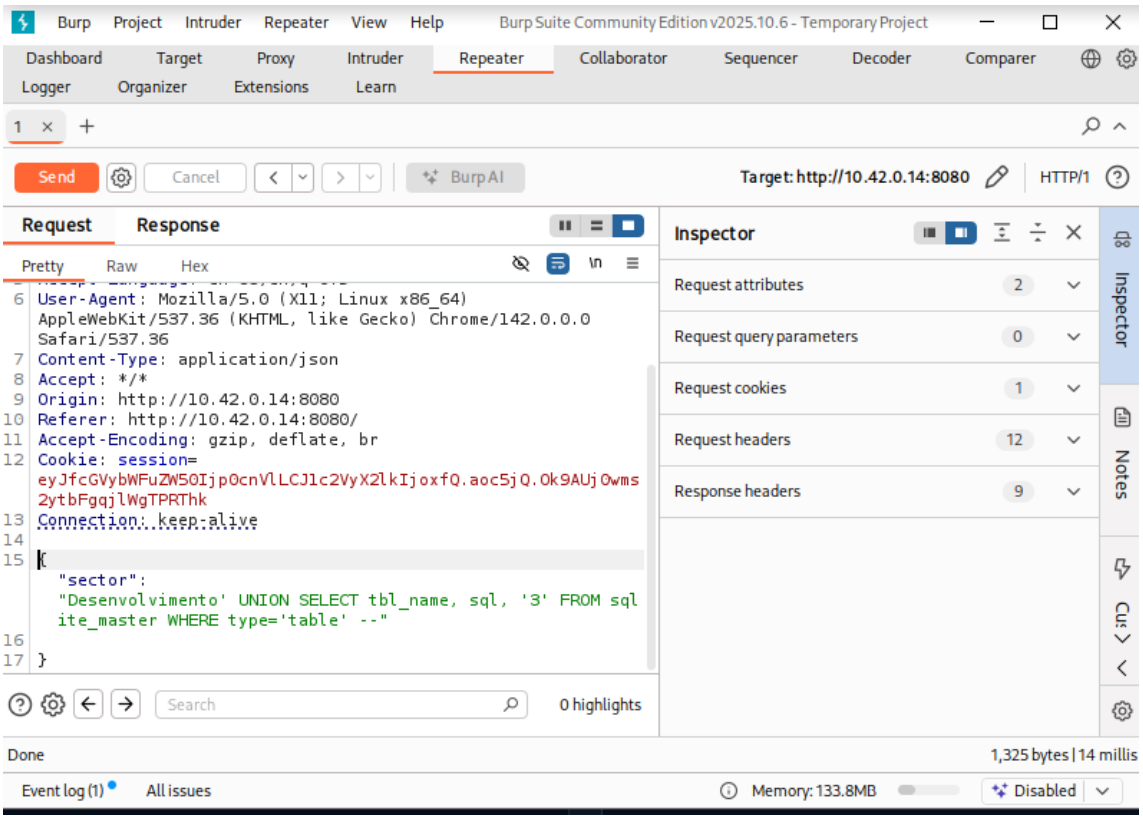


Na aba de setor, consultamos o setor, abrimos no repeater e enviamos o comando {"sector":"Desenvolvimento' OR '1'='1"}, quando enviamos, é descoberto que é, também vulnerável a SQL injection, pois subiu de 1 para três registros

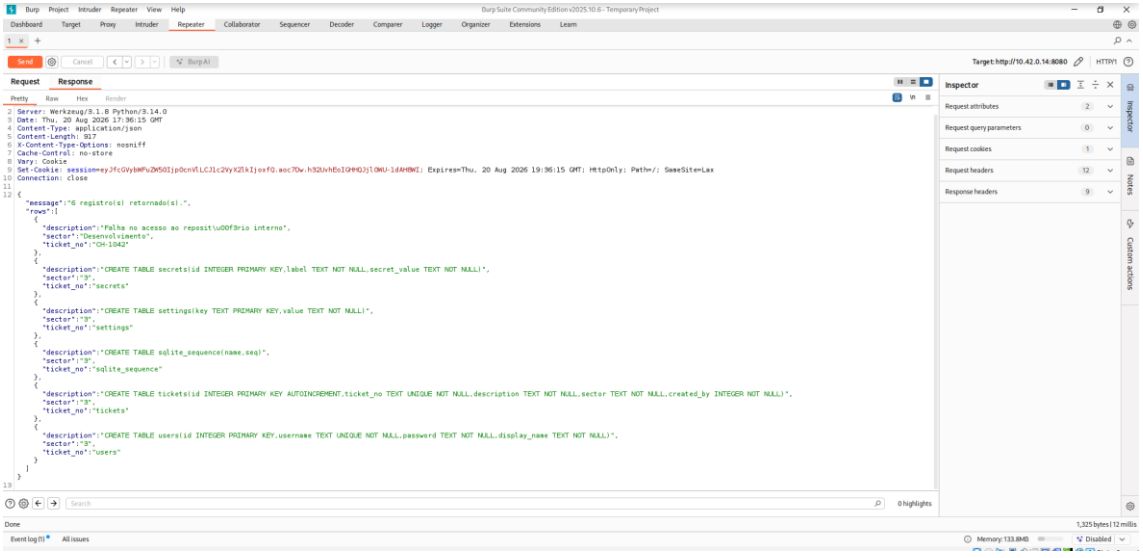




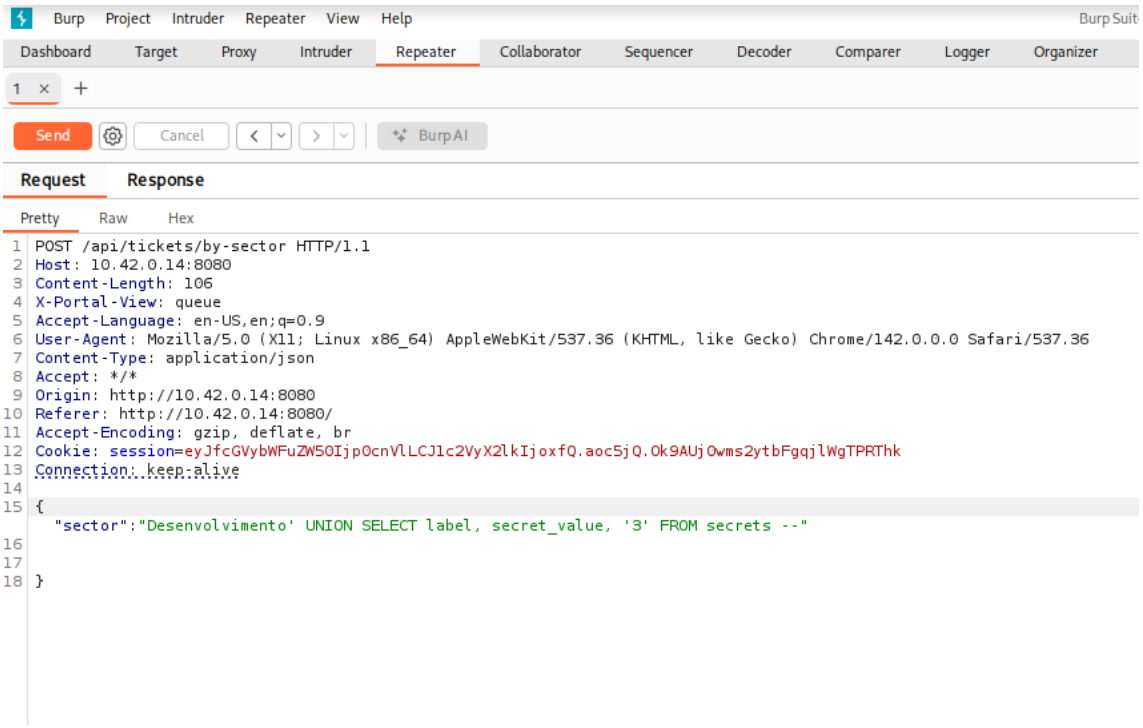
Depois, usamos o comando {"sector":"Desenvolvimento' UNION SELECT tbl_name, sql, '3' FROM sqlite_master WHERE type='table' --"}, os dois hífens no final, serve para tirar o ORDER BY que tinha sido programado, assim, conseguindo rodar o UNION SELECT.



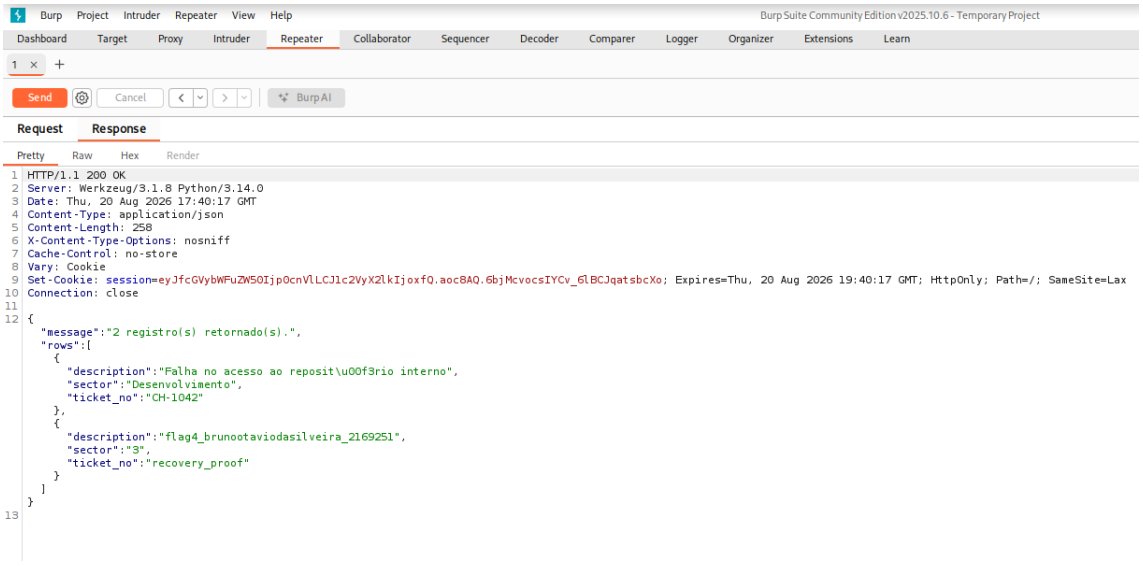
Com essa requisição, conseguimos descobrir as tabelas, as mais importantes sendo: tickets, users, settings e secrets, nela também, descobrimos as colunas, algo que será de extrema importância para a exploração



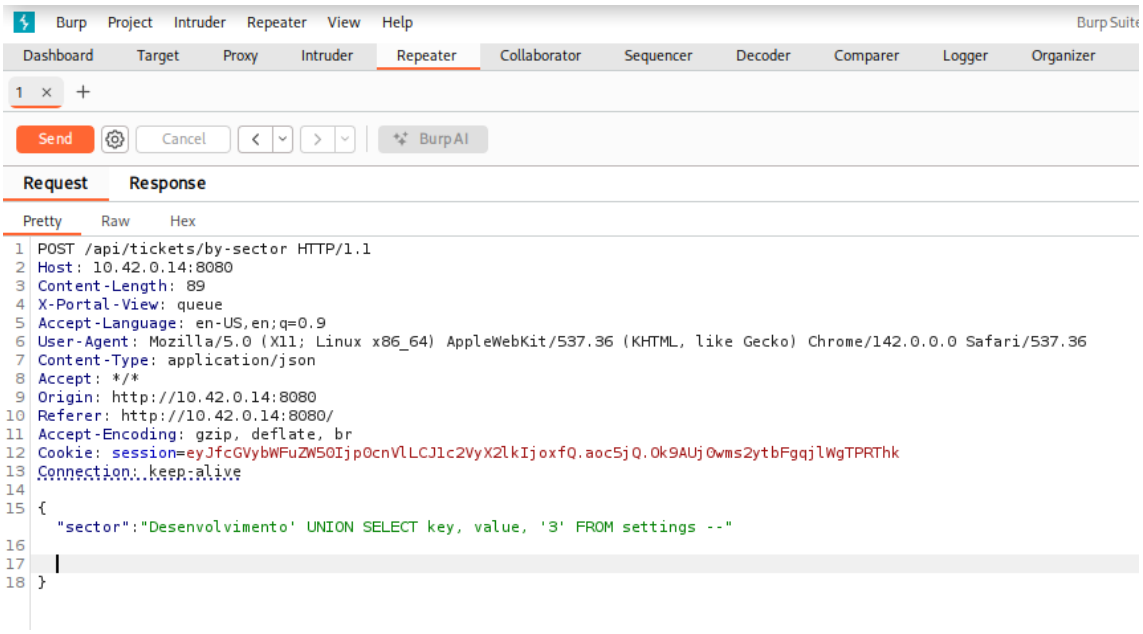
Primeiro, chegamos a tabela “secrets”(onde, geralmente, fica as flags) com o comando {"sector":"Desenvolvimento' UNION SELECT label, secret_value, '3' FROM secrets --"}



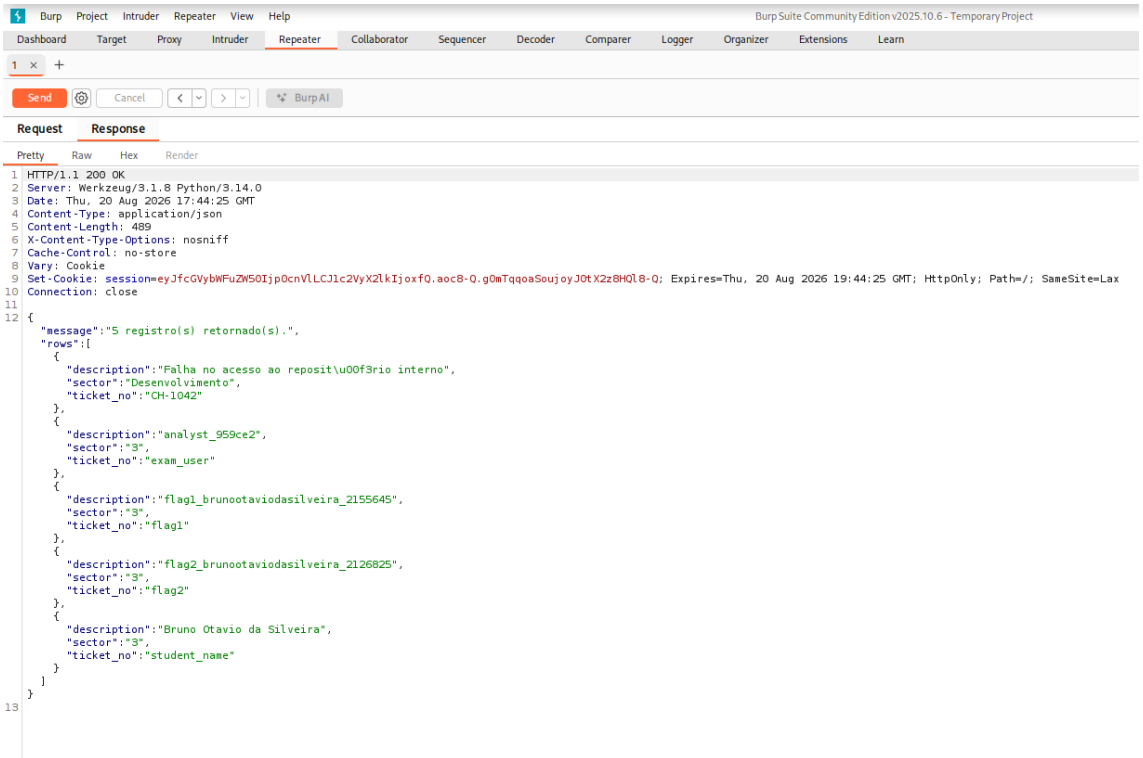
Na tabela, conseguimos descobrir a flag 4



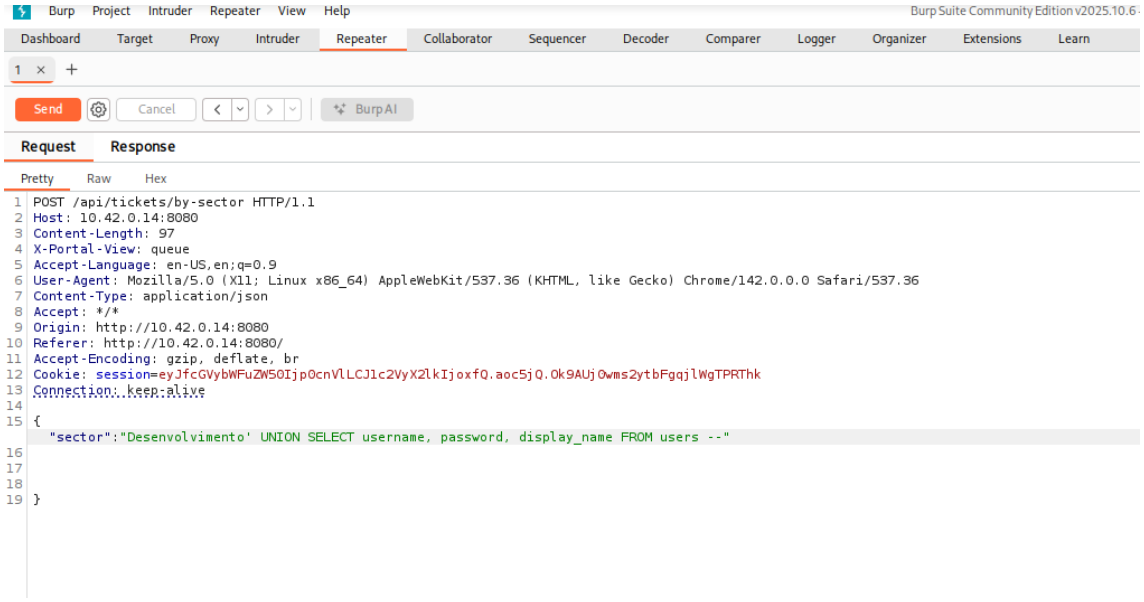
Depois disso, foi checado a tabela settings, que possui as colunas key e value, onde geralmente é escondido algo lá



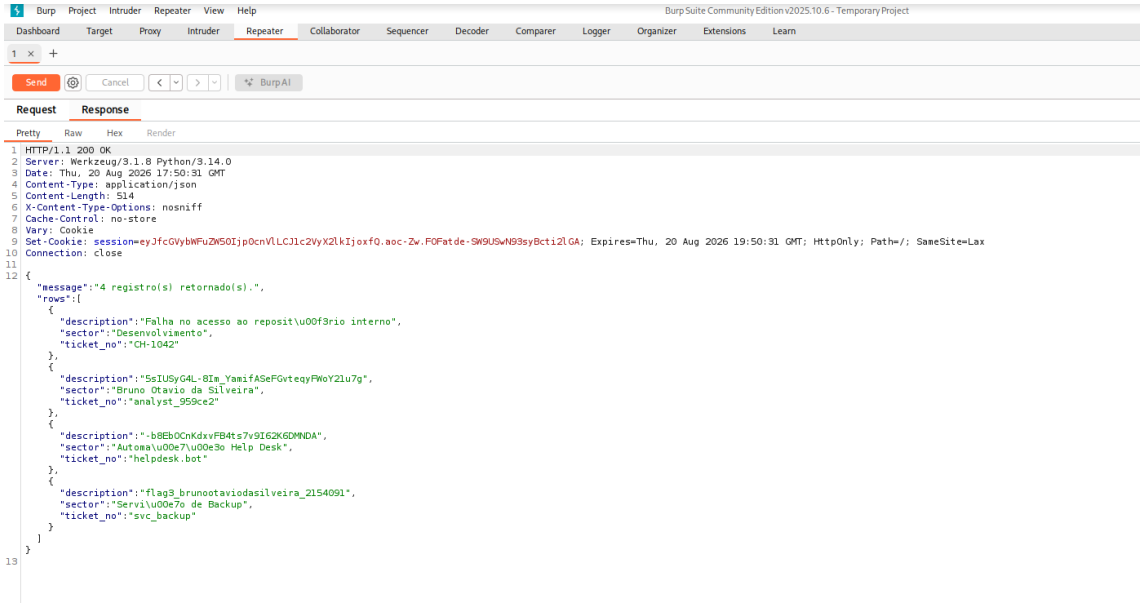
Lá, retornou algumas coisas que é guardada nessa coluna, inclusive a flag2



Por último, fomos checar a tabela users, que sabemos que tem as colunas username, password e display_name



Então, por fim, encontramos a flag 3



Observações finais: a tabela tickets apenas mostrou os chamados da aplicação, nada de muito relevante para a busca das flags

Agora, por fim, as vulnerabilidades:

Criticidade: **Alta**

Vulnerabilidade 1:

SQL Injection na Autenticação:

Criticidade: **Alta**

nela, o atacante consegue logar em qualquer conta sem saber a senha real

Como resolver: nunca deixar a senha salva em texto plano no banco de dados e também, entender tudo como texto, não como comando sql.

Vulnerabilidade 2: Divulgação de Informações em Comentários HTML:

Criticidade**:** Média (depende do conteúdo)

Lá, o programador tem que esconder comentários que possam comprometer e/ou facilitar a invasão de um atacante

Como resolver: apagando os comentários.

Vulnerabilidade 3: Validação Apenas no Frontend:

Criticidade: **Alta**

Nela, a validação acontece apenas no front-end, sendo interceptada facilmente (burp suíte, etc).

Como resolver: A validação PRECISA ser feita no back-end.

Vulnerabilidade 4: SQL Injection Baseado em UNION

Criticidade**: CRITÍCA**

O banco de dados é facilmente invadindo, vazando TODOS os dados da aplicação.

Como resolver: Da mesma forma da vuln1, deixar o texto ser lido apenas como texto, não como comando SQL